

Linux & DevSecOps Session 05 — Explained Simply

Sep 25, 2026 · @kiran

Welcome

This guide walks through every command and idea from your Session 05 mind map, explained in plain English — what it is, why it matters, and how to use it.

The flow: editing files in Vim → managing user accounts → logging in securely with SSH → safely offboarding an employee → finding files → installing software → managing background services → understanding network ports → managing running processes → and how it all ties into enterprise IT.

1. Vim Editor

Vim is a text editor you run inside the terminal. It has two main modes: **Normal mode** (for moving around and running commands) and a **colon mode** (`:`) for save/quit/search/replace actions.

Colon-mode commands (press `:` first)

Command	What it does
<code>:q</code>	Quit (only works if nothing changed)
<code>:wq</code> or <code>:x</code>	Save the file, then quit
<code>:q!</code>	Force quit and discard any changes
<code>:set nu</code> / <code>:set number</code>	Turn line numbers ON
<code>:set nonu</code>	Turn line numbers OFF
<code>/pattern</code>	Search forward for text
<code>?pattern</code>	Search backward for text
<code>:25</code> (any line number)	Jump straight to that line

Command	What it does
<code>3d</code>	Delete 3 lines starting from the cursor
<code>:%d</code>	Delete every line in the file
<code>:%s/old/new/g</code>	Find every "old" and replace it with "new"

Normal-mode commands (no `:` needed)

Command	What it does
<code>u</code>	Undo the last change
<code>Ctrl + r</code>	Redo (undo the undo)
<code>yy</code>	Copy ("yank") the current line
<code>p</code>	Paste the copied/cut line
<code>dd</code>	Cut (delete) the current line
<code>gg</code>	Jump to the top of the file
<code>G</code> or <code>Shift + g</code>	Jump to the bottom of the file

Tip: think of colon-mode as "file-level" actions (save, quit, search whole file) and normal-mode as "line-level" actions (copy, cut, undo one step).

2. User & Group Management

Every person (or service) that logs into a Linux server needs a **user account**. Accounts are often organized into **groups** so permissions can be handed out to many people at once instead of one by one.

User commands

Command	What it does
<code>useradd <username></code>	Creates a new user account

Command	What it does
<code>id <username></code>	Shows that user's ID number, group memberships, etc.
<code>passwd <username></code>	Sets or changes that user's password
<code>userdel -r <username></code>	Deletes the user AND removes their home folder (<code>-r</code> = remove)

Group management

Command	What it does
<code>usermod -g <group> <user></code>	Sets the user's primary group (their main/default group)
<code>usermod -aG <group> <user></code>	Adds the user to an extra ("secondary") group without removing them from others — the <code>-a</code> ("append") is important, or you'd wipe their other groups
<code>gpasswd -d <user> <group></code>	Removes the user from a secondary group

Key configuration files

File	What it stores
<code>/etc/passwd</code>	List of all user accounts on the system
<code>/etc/group</code>	List of all groups and their members
<code>/etc/sudoers</code> and <code>/etc/sudoers.d/</code>	Who is allowed to run commands as admin (<code>sudo</code>)
<code>/etc/ssh/sshd_config</code>	Settings for the SSH login service

Why it matters: almost every security and access-control task on Linux comes back to "which user, which group, which file."

3. SSH Key-Based Authentication

Instead of typing a password every time you log into a server, SSH keys let you log in with a matching pair of cryptographic keys — much more secure, since there's no password to guess or steal in transit.

The mechanism

- A **key pair** is generated: one **private** key and one **public** key. They're mathematically linked but you can't get one from the other.
- The private key **never leaves your computer**.
- The public key is shared with any server you want to log into.

Step 1 — Generate keys on the client (your laptop)

```
ssh-keygen -f <keyname>
```

This creates two files: `<keyname>` (private — keep secret) and `<keyname>.pub` (public — safe to share). You send the `.pub` file to the server admin.

Step 2 — Set up the server (done by the admin)

Step	Command / action
Create the user account	<code>useradd <username></code>
Create an <code>.ssh</code> folder in their home directory	<code>/home/<user>/.ssh</code>
Give the user ownership of it	<code>chown -R <user>:<user> .ssh</code>
Lock down folder permissions	<code>chmod 700 .ssh</code> (only the owner can access it)
Create the "allowed keys" file	<code>/home/<user>/.ssh/authorized_keys</code>
Paste the public key's contents into it	(manual step)
Lock down that file's permissions	<code>chmod 600 authorized_keys</code> (owner can read/write, nobody else)

Why the strict permissions? SSH refuses to trust a key file if it's too open to other users — it assumes it might have been tampered with.

Step 3 — Connect

```
ssh -i <private_key> <user>@<ip>
```

What happens behind the scenes (the authentication flow)

1. Your client contacts the server's IP on port 22 (SSH's default port).
2. The server looks in that user's home directory.
3. It opens the `.ssh` folder.
4. It reads the `authorized_keys` file.
5. It checks whether your private key mathematically matches any public key listed there.
6. Match found → you're logged in. No match → connection refused.

4. User Offboarding Workflow

When someone leaves a company (resignation or termination), their server access needs to be shut down carefully and in the right order. This protects the company from:

- **Sabotage** — a disgruntled employee doing damage before access is cut
- **Compliance failures** — auditors checking that ex-employees can't still log in

The 7 steps, in order

1. **Remove secondary groups** — `gpasswd -d <user> <group>`
2. **Reset their primary group** — `usermod -g <user> <user>` (every user needs at least one primary group, so you point it back to their own personal group)
3. **Lock the account** so it can't be used to log in:
 - `usermod -L <user>` — locks the password
 - `usermod -e 1 <user>` — sets the account to expire immediately
4. **Kick out any active sessions** they're still logged into:
 - `who` — see who's currently logged in
 - `pkill -u <user>` — force-close all of that user's processes/sessions
5. **Back up their home directory** before deleting anything:
 - `mkdir /backup`
 - `tar -czvf /backup/<file>.tar.gz /home/<user>`

6. **Search the whole system for leftover files** owned by them (in case they saved things outside their home folder): `find / -user <user>`

7. **Delete the account entirely** — `userdel -r <user>` (removes the account and home directory)

Note: this whole sequence can be automated via a shell script or through Single Sign-On (SSO) tooling, so it runs the same way every time instead of relying on someone remembering all 7 steps.

5. File Searching with `find`

`find` locates files anywhere on the system by name, type, owner, or other properties.

Basic syntax:

```
find <path> <options> <pattern>
```

Example	What it does
<code>find /</code>	Searches the entire filesystem, starting at the root
<code>find / -name "*.log"</code>	Finds every file ending in <code>.log</code> (wildcard match)
<code>find / -name "*sshd*"</code>	Finds files whose name contains "sshd" anywhere
<code>find / -type d -name "<dir_name>"</code>	Finds a directory (folder) with that name (<code>-type d</code> = directories only)
<code>find / -user <username></code>	Finds every file owned by that user

Why it's useful: this is exactly the command used in offboarding step 6 above — to make sure no stray files owned by a departed employee are left behind.

6. Package Management with `dnf`

A "package manager" installs, updates, and removes software for you — instead of manually downloading and compiling programs.

- **dnf** (Dandified YUM) is the package manager on **RedHat Enterprise Linux** and **Amazon Linux**.
- **yum** still works too — it's just a soft link (shortcut) that points to `dnf`.

Repository configuration

Packages come from "repositories" (repos) — online (or internal) software catalogs.

- Repo config files live in `/etc/yum.repos.d/`, each with a `.repo` extension.
- Large companies often run their **own offline repos** — local mirror servers or proxy networks — instead of reaching the public internet, for security and speed.

Common `dnf` commands

Command	What it does
<code>dnf install <package></code>	Installs a package
<code>dnf install <package>:<version></code>	Installs a specific version
<code>dnf install -y <package></code>	Installs without asking "are you sure?" (<code>-y</code> = yes to all prompts)
<code>dnf remove <package></code>	Uninstalls a package
<code>dnf update <package></code>	Updates a package to the latest version
<code>dnf search <keyword></code>	Searches for packages matching a keyword
<code>dnf info <package></code>	Shows details about a package
<code>dnf list installed</code>	Lists everything currently installed
<code>dnf repolist</code>	Lists which repositories are configured

Ubuntu equivalent: on Debian-family systems (like Ubuntu), the same job is done by `apt` / `apt-get` instead of `dnf`.

7. Service Management with `systemctl`

A **daemon** is a program that runs continuously in the background (not tied to any terminal session) — examples: `sshd` (SSH login service), `nginx` and `httpd` (web servers).

`systemctl` ("System Control") is the tool used to manage these background services.

Command	What it does
<code>systemctl start <service></code>	Starts the service right now

Command	What it does
<code>systemctl stop <service></code>	Stops the service right now
<code>systemctl restart <service></code>	Stops then starts it again (useful after a config change)
<code>systemctl status <service></code>	Shows whether it's running, and recent log lines
<code>systemctl enable <service></code>	Makes it auto-start every time the server boots
<code>systemctl disable <service></code>	Turns off auto-start on boot

Why "enable" matters: if a server reboots (crash, patching, power loss) and the service isn't "enabled," it won't come back on its own — causing an outage until someone notices and starts it manually.

8. Network Management & Ports

Think of a **port** as a numbered door or apartment number on a server — it lets many different services share one network address (IP) without getting mixed up. Ports range from **0 to 65535**.

Standard default ports

Port	Used for
22	SSH (remote login)
80	HTTP (plain web traffic)
443	HTTPS (encrypted web traffic)
3306	MySQL database
8080	Jenkins / general web applications
25	SMTP (email sending)
53	DNS (domain name lookups) — also what AWS Route 53 is named after

Checking what's listening

```
netstat -lntp
```

Flag	Meaning
-l	Show only ports currently listening
-n	Show addresses/ports as numbers, not names
-t	Show TCP connections
-p	Show the process ID using each port

Troubleshooting connectivity, step by step

1. **Is the service even running?** → `systemctl status <service>`
2. **Is it listening on the port locally?** → `netstat -lntp`
3. **Is the process actually alive?** → `ps -ef | grep <service>`
4. **Is an external firewall blocking it?** → check things like an AWS Security Group's inbound rules (e.g., does `0.0.0.0/0` — meaning "any IP" — have access to the right port?)

This order matters: check locally first (service → port → process), then check the network layer around it (firewall rules) — that way you don't waste time debugging firewall rules for a service that was never running in the first place.

9. Process Management with `ps` & `kill`

Every running task on Linux — from a shell command to a background service — is a **process**. Processes have a parent-child relationship: one process can spawn (start) another.

- **PID** = Process ID (a unique number for that running task)
- **PPID** = Parent Process ID (the ID of whatever process started it)

Viewing processes

Command	What it does
<code>ps</code>	Shows processes running in your current shell only
<code>ps -ef</code>	Shows every process on the whole system
<code>ps -ef grep <name></code>	Filters that list down to processes matching a name

Foreground vs. background

- **Foreground process** — locks your terminal until it finishes. Example: `sleep 10` (you have to wait 10 seconds before you can type anything else).
- **Background process** — runs without blocking you, started by adding `&` at the end. Example: `sleep 60 &` (you get your terminal back immediately while it runs).

Terminating processes

```
kill -u <username>
```

Kills **every** process belonging to that user — this is the same command used in the offboarding workflow (Step 4) to force-close a departed employee's active sessions.

10. Enterprise Systems & Career Insights

Everything above works on one server managed by hand. Large companies need it to work across thousands of servers and employees — that's where these systems come in:

- **Single Sign-On (SSO)** — log in once, get access to many systems, instead of separate passwords everywhere
- **CyberArk** — manages and vaults privileged (admin-level) credentials so they aren't scattered around
- **Azure AD / Active Directory** — Microsoft's system for centrally managing user accounts and permissions across a whole organization
- **SFTP jump hosts** — a single, tightly controlled server you must pass through before reaching more sensitive internal servers

Why bother learning raw Linux commands?

Modern tools like **Docker**, **Kubernetes**, and **Terraform** are built as layers *on top of* Linux — they automate and abstract away a lot of this manual work. But when something breaks, troubleshooting almost always means dropping back down to the underlying Linux system: checking a process, a port, a log file, or a permission. Knowing the fundamentals in this guide is what lets you actually fix problems instead of just clicking around a dashboard.